

哈尔滨工业大学

编译系统实验报告（二）

姓名	曹云涵
学号	2021113258
学院	未来技术学院
教师	单丽莉

2024 年 4 月

一、实验目的

1. 巩固对语义分析的基本功能和原理的认识。
2. 能够基于语法指导翻译的知识进行语义分析。
3. 理解并处理语义分析中的异常和错误。

二、实验环境

1. WSL2-Ubuntu_22.04
2. GCC 11.4.0
3. Flex 2.6.4
4. Bison 3.8.2

三、实验过程

1. 实验内容

本次实验完成了对于 C—语言的语义分析，对于语义错误的检测。

2. 数据结构

本次实验基于实验 1，继续使用语法分析树作为数据结构。此外，我们引入了两种新的数据结构：变量类型 Type 和字符表中的域 FieldList。这些结构的实现遵循了参考书中的指定方法，详细实现在此不再详述。

3. 语义分析过程

在语义分析阶段，我们假设词法和语法无误，首先进行词法和语法分析以构建语法树。之后，通过遍历语法树来实现语义分析。遍历过程中，针对不同的错误类型，我们将这些错误映射到语法树的相应部分，以此作为错误判断的依据。这样，当遇到相同情况时，我们可以据此评判错误。

下面展示一些错误的情况的判定方法：

(1) 错误类型 13：对非结构体类型变量使用“.”操作符。

```
if(strcmp(fir_bro->Token,"ASSIGNOP")==0){
    //Exp + Exp ASSIGNOP Exp
    //child->ID | Exp LB Exp RB | Exp DOT ID
    Treenode child_child=child->firstchild;
    if(child_child==NULL) return NULL;
    else if(strcmp(child_child->Token,"ID")==0&&child_child->nxtbro==NULL);
    else if(strcmp(child_child->Token,"Exp")==0&&strcmp(child_child->nxtbro->Token,"LB")==0);
    else if(strcmp(child_child->Token,"Exp")==0&&strcmp(child_child->nxtbro->Token,"DOT")==0);
    else{
        printf("Error type 6 at Line %d: The left-hand side of an assignment must be a variable.\n",now->line);
        return NULL;
    }
    Type left_type=Exp(child);
    Type right_type=Exp(fir_bro->nxtbro);
    if(typecmp(left_type,right_type)==0) return right_type;
    else {
        printf("Error type 5 at Line %d: Type mismatched for assignment.\n",now->line);
        return NULL;
    }
}
```

在本实验中, 首先会定位到语法单元“.”。接下来, 我们将在语法分析树中对其进行回溯, 以确定前一个语法单元是否定义为结构体。如果定义为结构体, 则该用法是合法的; 如果不是, 则构成非法调用。由于采用二叉树来实现多叉树, 语法单元“.”及其前一个语法单元在树中是兄弟节点, 且前一个单元位于“.”的左侧。因此, 我们只需检查该节点的类型是否为 struct 即可判断其合法性。

(2) 错误类型 1: 变量在使用时未经定义。

```
else if(strcmp(child->Token,"ID")==0){
    //Exp → ID
    //printf("aExp → ID\n");
    Vari find_id=find_vari_table(child->Id_Type);
    if(find_id==NULL|| find_id->is_def_struct == DEF_IN_STRUCT){
        printf("Error type 1 at Line %d: Undefined variable \"%s\".\n",now->line,child->Id_Type);
        return NULL;
    }
}
```

当`strcmp`函数的返回值为 0 时, 表明比较的两个字符串相等, 这意味着当前的语法单元是一个 ID 类型。接下来, 我们在相应的符号表中查找此变量名是否存在。存在一种特殊情况, 即变量定义在结构体内部, 虽然在符号表中可以找到此变量, 但这种情况通常是不合法的, 因此需要进行特殊处理。

如果变量未在符号表中找到, 或者其定义在结构体内, 我们应返回“Error Type 1”以指示错误。

(3) 错误类型 5, 6: 赋值号两边类型不匹配, 赋值号左边出现只有右值的表达式。

针对这两种错误, 我们需要检索的语法单元是“=”, 也就是 ASSIGNOP。我们需要分别检查 ASSIGNOP 的父节点 (赋值号左边的内容) 以及 ASSIGNOP 的左子节点 (赋值号右边的内容), 原因同上, 我们采用的是二叉树表示多叉树。

针对错误类型 5: 我们需要判断左侧是否是 INT 或者 FLOAT, 如果是, 那么这个赋值语句就是形如“1=ax+b”的, 那么就是满足错误类型 5 的。

针对错误类型 6: 我们需要判断两侧 id 的 basic 是否相同, 如果不相同就证明两侧的类型是不匹配的, 就要返回错误类型 6。

(4) 错误类型 12: 数组访问操作符“[...]”中出现非整数。

每当我们遇到数组访问语句的时候, 我们需要对于语法单元进行拆分, 以 a[1]为例: 我们会将其拆分为 4 个语法单元: “a”, “[”, “1”, “]”。那么显然这种错误对应就是第三个语法单元对应的类型不是 INT 而是 FLOAT。

```
Type right_type=Exp(child->nxxtbro->nxxtbro);
if(right_type==NULL) return NULL;
if(right_type->type==BASIC&&right_type->basic==T_INT){
    return left_type->array.elem;
}
else{
    // [float]
    printf("Error type 12 at Line %d: Not integer.\n",child->line);
    return NULL;
}
}
```

在此实验中, 若`if`语句中第三个语法单元为整数类型, 则将其当作数组处理。相对地, `else`语句中若第三个语法单元是 FLOAT 类型, 则需要返回错误类型 12。通过这种方式, 我们分解并分析每一种潜在的错误情况, 具体针对可能出错的语法单元进行识别和判断。

四、实验的操作流程

编写了 MakeFile 文件用来进行实验的运行, 其中设置了对应语言使用的编译环境以及一些伪目标。文件内容如下:

```

# 编译目标: src目录下的所有.c文件
CFILES = $(shell find ./ -name "*.c")
OBJ5 = $(CFILES:.c=.o)
LFILE = $(shell find ./ -name "*.l")
YFILE = $(shell find ./ -name "*.y")
LFC = $(shell find ./ -name "*.l" | sed s/[^\.]*\\.l/lex.yy.c/)
YFC = $(shell find ./ -name "*.y" | sed s/[^\.]*\\.y/syntax.tab.c/)
LFO = $(LFC:.c=.o)
YFO = $(YFC:.c=.o)

parser: syntax $(filter-out $(LFO),$(OBJ5))
    $(CC) -o parser $(filter-out $(LFO),$(OBJ5)) -lfl

syntax: lexical syntax-c
    $(CC) -c $(YFC) -o $(YFO)

lexical: $(LFILE)
    $(FLEX) -o $(LFC) $(LFILE)

syntax-c: $(YFILE)
    $(BISON) -o $(YFC) -d -v $(YFILE)

-include $(patsubst %.o, %.d, $(OBJ5))

# 定义的一些伪目标
.PHONY: clean test1 test3 test5 test6 test7 test10 test12 test2 test4 test8 test9 test11 test13 test14 test15 test16 test17
test1:
    ./parser ../test1.cmm
test3:
    ./parser ../test3.cmm
test5:
    ./parser ../test5.cmm
test6:
    ./parser ../test6.cmm
test7:
    ./parser ../test7.cmm
test10:
    ./parser ../test10.cmm

```

在运行项目的时候首先使用"make parser"命令构建语义分析器，然后使用命令"./parser test1.cmm"来完成对于 test1.cmm 文件的语义分析。

五、 实验的运行结果（仅展示部分）

```

cyh173702@Lenovocyh:~/temp/code_lab2$ ./parser test1.cmm
Error type 1 at Line 4: Undefined variable "j".
Error type 5 at Line 4: Type mismatched for assignment.
cyh173702@Lenovocyh:~/temp/code_lab2$ ./parser test2.cmm
Error type 2 at Line 4: Undefined function "inc".
cyh173702@Lenovocyh:~/temp/code_lab2$ ./parser test4.cmm
Error type 4 at Line 6: Redefined function "func".
cyh173702@Lenovocyh:~/temp/code_lab2$ ./parser test5.cmm
Error type 5 at Line 4: Type mismatched for assignment.
cyh173702@Lenovocyh:~/temp/code_lab2$ ./parser test6.cmm
Error type 6 at Line 4: The left-hand side of an assignment must be a variable.
cyh173702@Lenovocyh:~/temp/code_lab2$ ./parser test7.cmm
Error type 7 at Line 4: Type mismatched for operands.
cyh173702@Lenovocyh:~/temp/code_lab2$ ./parser test8.cmm
Error type 8 at Line 4: Type mismatched for return.
cyh173702@Lenovocyh:~/temp/code_lab2$ ./parser test11.cmm
test11.cmm: No such file or directory
cyh173702@Lenovocyh:~/temp/code_lab2$ ./parser test12.cmm
Error type 12 at Line 4: Not integer.
Error type 5 at Line 4: Type mismatched for assignment.
cyh173702@Lenovocyh:~/temp/code_lab2$ ./parser test13.cmm
Error type 13 at Line 9: Illegal use of ".".
cyh173702@Lenovocyh:~/temp/code_lab2$ █

```

六、 实验总结

本次实验加深了我对语法分析树构建及其功能的理解，并使我能够回顾并深化对前次实验的认识。我学会了从根本上识别和处理语义分析中的错误，即通过将语句分解为一系列语法单元，并检查每个单元是否有误以及错误的具体类型。此外，这次实验也帮助我实践和加固了课堂上学到的知识，完成了对 C—语言的基础语义分析任务。